

Opening the Black Box of LLM-powered Semantic Query Processing

Kangkang Qi
Beijing Institute of Technology
qikangkang0126@gmail.com

Zhuoyan Wei
Beijing Institute of Technology
weizhuoyan2025@gmail.com

Wenbo Li
Wuhan University
liwenbo82@whu.edu.cn

Dongyang Xie
Wuhan University
xdy1122310013@163.com

Hao Zhang
The Chinese University of Hong Kong
zhanghaowuda12@gmail.com

Yuanyuan Zhu
Wuhan University
yyzhu@whu.edu.cn

Jeffrey Xu Yu
HKUST (Guangzhou)
jeffrexyuyu@hkust-gz.edu.cn

Kangfei Zhao
Beijing Institute of Technology
z kf1105@gmail.com

ABSTRACT

LLM-powered semantic operators, such as semantic filter, projection, and join, enable users to express complex analytics intents over structured and unstructured data. However, they introduce new challenges in controllability and transparency. Unlike classical relational operators, the execution behavior of semantic operators is non-deterministic and context-dependent, and the optimization requires navigating complex trade-off among latency, monetary cost, and result quality, which remains largely opaque to end users. In this demonstration, we present SEMASTUDIO, an interactive visual interface built on top of SEMA, a high-performance semantic query engine extending DuckDB. SEMASTUDIO demystifies the end-to-end LLM-driven query processing pipeline by making internal optimizations transparent and tunable. It provides a clear view of logical LLM-assisted optimizations, visualizing the step-by-step processes of expression compression and predicate deduction alongside runtime metrics. Furthermore, SEMASTUDIO exposes the internal workflows of Adaptive Query Execution (AQE). It profiles candidate physical execution paths, incorporating operator reordering, fusion, and prompt batching, and renders a latency-cost Pareto frontier to help users balance trade-offs under specified quality constraints. By exposing these opaque internal mechanics, SEMASTUDIO bridges the gap between powerful LLM-powered optimizations and the interpretability required by modern database users and system developers.

PVLDB Reference Format:

Kangkang Qi, Zhuoyan Wei, Wenbo Li, Dongyang Xie, Hao Zhang, Yuanyuan Zhu, Jeffrey Xu Yu, and Kangfei Zhao. Opening the Black Box of LLM-powered Semantic Query Processing. PVLDB, 14(1): XXX-XXX, 2020. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/BITQiKangK/SemaStudio.git>.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Recent advances in large language models (LLMs) have catalyzed the emergence of LLM-powered analytics over both structured and unstructured data. A variety of systems [1–4] support semantic queries expressed in DataFrame primitives or SQL user-defined functions (UDFs), where a core set of semantic operations, such as semantic filter, projection, join, and aggregation, are composed with relational operations. Unlike relational operators with deterministic semantics, stable algebraic laws, and well-understood cost profiles, semantic operators are inherently non-deterministic and context-dependent: their execution behavior depends on the NL expression, model choice, prompting strategy, and external LLM serving environments. They also introduce two new optimization dimensions, monetary cost and result quality, which are absent from classical query processing. As a result, optimizing semantic queries requires decision-making about complex trade-off among latency, cost, and quality. These challenges call for new system-level optimizations beyond traditional cost-based optimization.

Existing systems have proposed various optimization strategies tailored for semantic operators, concentrating on reducing the overhead of LLM invocations, including operator fusion, prompt optimization and batching, and adaptive query processing. While these techniques can yield significant efficiency gains, they provide limited visibility into when and why a particular optimization is triggered, or how it affects the plans and query evaluation. In practice, every change to LLM choice, prompting strategy, batch size for prompt batching, and predicate order can lead to different query results regarding latency, cost, and quality. Without tool support for inspecting intermediate optimization steps and comparing query plans, users and system developers have to treat the entire pipeline as a black box, making it difficult to diagnose unexpected results, tune configurations, or build trust in the system’s decisions.

In this demonstration, we present SEMASTUDIO, an interactive graphical interface built on top of SEMA [4], a high-performance semantic query engine that extends DuckDB [5] with semantic operators as first-class citizens. SEMA introduces SEMASQL, a declarative SQL dialect that injects NL expressions into standard SQL clauses, enabling seamless composition of semantic and relational computation. In addition, it provides end-to-end optimization through

logical-level LLM-assisted expression compression and predicate deduction, and runtime optimizations via adaptive query execution (AQE) that combines operator fusion and prompt batching to find Pareto-optimal execution paths balancing latency and cost under user-specified quality constraints.

SEMASTUDIO complements SEMA by exposing the full query processing pipeline, from system configuration to optimization and adaptive execution, through a rich interactive visual interface. Specifically, SEMASTUDIO supports the following capabilities: ① *Flexible System Configurations*. Users can interactively configure LLM choices, batch size, quality constraints, and AQE policies, and immediately observe how each change influences the optimizer’s decisions and execution behavior. ② *Transparent LLM-assisted Optimization*. SEMASTUDIO visualizes two LLM-assisted expression optimizations, displaying the original and rewritten NL expressions side by side, along with per-step runtime and cost metrics, so that users can trace every optimization decision to its effect on the transformed plan. ③ *Adaptive Execution Profiling*. The interface exposes all candidate execution paths generated during AQE, streams their runtime statistics, and renders the Pareto frontier in a latency–cost scatter plot, revealing how different plan choices affect end-to-end performance and cost. ④ *Iterative Evaluation*. SEMASTUDIO enables an ‘edit-and-rerun’ workflow: users can modify configurations and resubmit queries to compare different plans and results across different settings, making LLM-powered semantic query processing easily tunable. The demonstration showcases these capabilities through realistic end-to-end scenarios involving multiple semantic filters SEMASQL queries, illustrating how SEMASTUDIO bridges the gap between powerful but opaque LLM-driven optimization and the transparency needed by database users and system developers.

2 SYSTEM ARCHITECTURE

2.1 Query Syntax & System Overview

Query Syntax for Semantic Processing. We introduce SEMASQL, a declarative SQL dialect that extends standard SQL with Natural Language (NL) expressions to support semantic query processing. The syntactic cornerstone is an NL expression with column placeholders, written as $s' \dots \{ \dots \}$, where each $\{ \}$ refers to a column. By injecting NL expressions into SQL clauses, e.g., SELECT for semantic projection, WHERE/HAVING for semantic filtering, and JOIN ON for semantic joining, SEMASQL implicitly defines a core set of semantic operators: *SemFilter*, *SemProj*, *SemJoin*, *SemOrderBy*, and *SemAgg*. At runtime, these NL expressions are evaluated by an LLM that performs the corresponding semantic reasoning for each tuple. SEMASQL remains fully compatible with SQL, supporting complex constructs such as common table expressions, window functions, and subqueries. This design allows users to seamlessly compose relational and semantic computation within a single query. Fig. 1 illustrates an exemplar SEMASQL query that joins two tables and applies three *SemFilter* predicates in the WHERE clause to identify user reviews that are informative, positive, and suitable for kids.

System Overview. Fig. 2 depicts an overview of the system, comprising SEMA, the back-end semantic query engine built on DuckDB[5], and SEMASTUDIO, the front-end interactive visualization platform. A SEMASQL query submitted through SEMASTUDIO is processed in

three stages: parsing and planning, optimization, and execution. In the first stage, the parser and planner identify NL expressions within the query to produce an initial logical plan where semantic operators are first-class plan nodes alongside relational operators. In the optimization stage, SEMA applies two LLM-assisted NL expression optimizations for reducing the overhead of LLM invocation, prior to conventional rule-based optimizations for standard SQL. In the execution stage, SEMA introduces physical optimizations tailored to semantic operators, including semantic operator fusion and prompt batching. Since runtime characteristics of semantic operators are difficult to estimate statically, SEMA further employs adaptive query execution (AQE) to dynamically explore candidate execution paths over a small data sample and select a Pareto-optimal path that balances latency and cost under a user-specified quality constraint. The selected path is then applied to process the remaining data at scale. On the front-end, SEMASTUDIO provides system configuration panels, displays the procedure and results of LLM-assisted expression optimizations, and profiles and compares the runtime characteristics of different paths through a dedicated graphical user interface, which will be detailed in §3.

2.2 SEMA on DuckDB

Optimizer. SEMA’s optimizer focuses on reducing the cost of LLM invocations without compromising accuracy. Since users may write arbitrary NL expressions in SEMASQL, NL expressions are inherently verbose, redundant, or ambiguous, leading to unnecessary overhead during LLM evaluation. To address this, the optimizer introduces two LLM-assisted rewrite rules for optimizing these NL expressions in the local plan.

Expression Compression. For each NL expression, the optimizer first invokes an auxiliary LLM with Chain-of-Thought (CoT) prompting to compress and refine the instruction in the expression. The CoT prompt includes decomposed instructions for removing stop words, simplifying sentence structure, and consolidating repeated references to placeholders. Then, the optimizer invokes a second auxiliary LLM to verify the semantic equivalence between the compressed and original expressions. This step requires only two auxiliary LLM calls for all NL expressions while potentially reducing per-tuple inference cost and improving semantic clarity for query evaluation.

Predicate Deduction. After compression, the optimizer seeks opportunities to deduce symbolic necessary conditions from NL predicates in *SemFilter* operators, rewriting them as relational filters that can be pushed down to prune tuples before expensive LLM evaluation. SEMA employs a two-stage LLM workflow for this purpose. First, a Deduction LLM receives the NL predicates together with database schema metadata and column statistics, and generates candidate SQL predicates via CoT prompting. Second, a Reflection LLM verifies each candidate by checking whether it is a sound necessary condition of the original semantic expressions. Only verified predicates are retained and incorporated into the optimized plan. This back-forward self-reflection mechanism mitigates LLM hallucination and preserves query correctness.

Executor. SEMA supports fusing consecutive semantic operators (e.g., two *SemFilter* on the same column) into a single operator

```

-- Find all valid and positive user reviews for
-- the 'FAMILY' category apps that indicate the
-- app is suitable for children
SELECT *
FROM user_reviews ur
JOIN playstore p
ON ur.app = p.app
WHERE p.category = 'FAMILY'
AND s'{translated_review} is a complete
sentence and has informative content'
AND s'{translated_review} is a positive user
review'
AND s'{translated_review} shows that this app
is suitable for kids';

```

Figure 1: An Exemplar SEMASQL Query

whose NL expressions are concatenated, thereby reducing the number of LLM invocations per tuple. Complementarily, prompt batching packs multiple tuples into a single LLM prompt to improve throughput and amortize per-call overhead. Since fusion, batching, and reordering of *SemFilter* operators affect latency, financial cost, and accuracy in ways that can hardly be estimated before runtime, SEMA postpones the decisions of the physical plan to runtime by Adaptive Query Execution (AQE). The AQE framework operates in three phases: it first collects per-predicate selectivity and correlation statistics on a small data sample, then generates and micro-evaluates candidate execution paths, i.e., reordering, fusion, and batching candidate paths, and finally selects a Pareto-optimal path balancing latency and token cost under a user-specified accuracy threshold. The selected path is applied to process the remaining data at scale.

While these optimizations can significantly reduce execution overhead, their effects are influenced by prompt design, data distribution, model choice, and inference environment, making the optimization process largely opaque. To enable in-depth inspection and understanding of the optimization and execution pipeline in SEMA, we develop SEMASTUDIO, a graphical interface that supports system configuration and provides visual analysis of every stage of the pipeline for both developers and end users.

2.3 SEMASTUDIO

SEMASTUDIO is designed to make the system environment, query workflow, and the opaque internal optimization and execution pipeline transparent and accessible. A rich set of runtime parameters is essential in SEMA because NL expressions exhibit substantial complexity and variability, making optimization and execution behavior highly sensitive. SEMASTUDIO allows users to configure these runtime parameters interactively, including LLM choices and inference service endpoint URLs for both SEMA’s optimizer and executor, as well as AQE-related options such as quality constraints and execution policies (e.g., latency-first or cost-first), etc. Notably, the system supports an OpenAI-compatible inference framework, ranging from remote LLM service platforms such as *OpenRouter* to local inference engines like *vLLM*, enabling seamless connection to diverse model serving backends. With these configurations in place, users can interactively observe how different runtime parameters affect system behavior, not only through final query results,

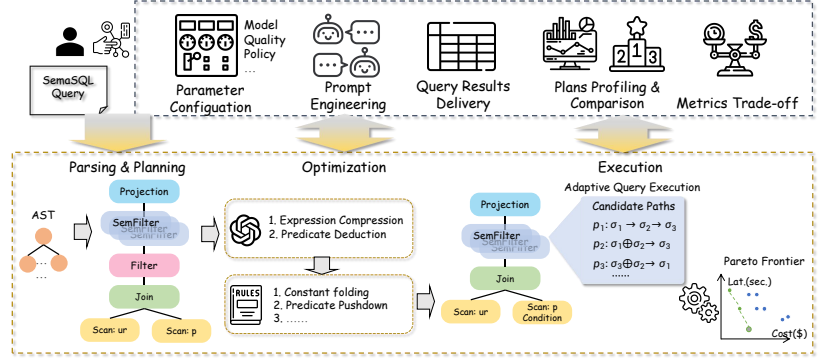


Figure 2: The Overview of SEMA and SEMASTUDIO

execution latency, and monetary cost, but also via rich intermediate information exposed throughout the query processing workflow, such as the outputs, latency, and cost of LLM-assisted optimization rules and the execution path selection process during AQE. To support this interactive experience, SEMASTUDIO communicates with the backend SEMA system through HTTP requests and responses implemented with Flask, a lightweight web application framework: the front-end submits user queries together with runtime configuration parameters and receives JSON responses containing both final outputs and detailed intermediate information from the optimization and execution stages.

3 DEMONSTRATION

We present the running example in Fig. 1 to showcase the user interaction interfaces of SEMASTUDIO, as illustrated in Fig. 3. The left panel (Fig. 3a) provides the input/output interface for composing and submitting queries, configuring execution parameters, and inspecting query results, while the right panel (Fig. 3b) visualizes the optimization and adaptive execution process in the system.

3.1 Input, Output and Configuration

Fig.3a illustrates the main input/output interface of SEMASTUDIO. The workflow begins with the user connecting to a DuckDB database file or other compatible data source through the connection panel in Fig. 3a-①. Once a connection is established, the system automatically loads and displays the available schemas, i.e., table names, column names, and types, for user inspection as shown in Fig. 3a-②. Users can browse the schema to compose a SEMASQL query, which is especially helpful when writing NL expressions over unstructured columns. User-specific SEMASQL query is issued in the SQL editor in Fig. 3a-③, e.g., the running example in Fig. 1.

The central control component is the configuration panel in Fig. 3a-④. Through this panel, users can specify the underlying LLM endpoint and API parameters, batch size for batch prompting, choose AQE settings such as tolerance to result quality, latency-first or cost-first policy, enable or disable individual optimization rules such as predicate deduction, and adjust other execution options. Once the setting is finished, users can click **EXPLAIN** to swiftly inspect the plan before execution, which is the plan in the bottom-right of Fig. 2 after the specific expression optimizations and the standard optimizations of DuckDB for relational operators.

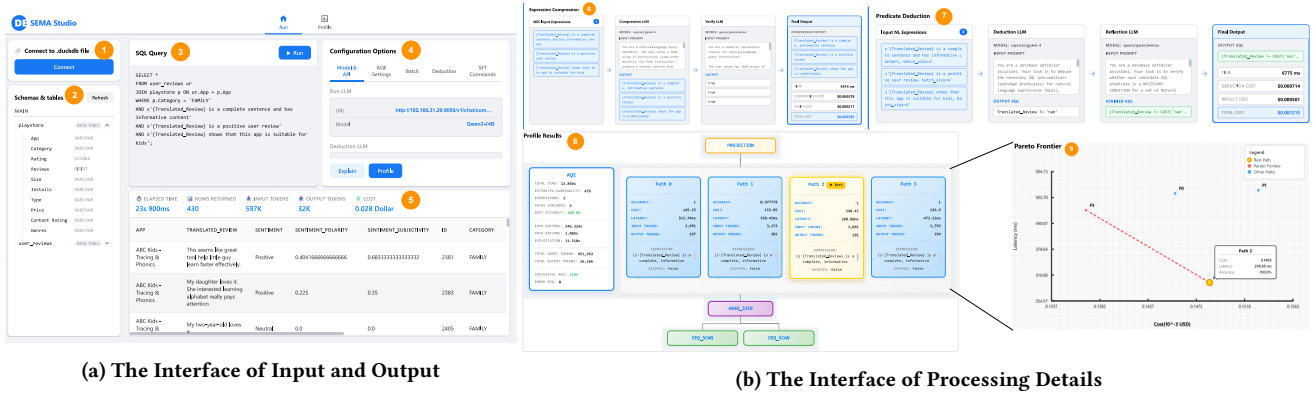


Figure 3: Demonstration of a Running Example SEMASQL Query

When the user clicks **RUN** in Fig. 3a-③, the system starts the query execution and will return the final query result as shown in Fig. 3a-⑤. The result table presents the returned tuples in ordinary relational form, enabling users to inspect the answer in a familiar database interface. In addition, SEMASTUDIO reports high-level result statistics such as elapsed time, the number of returned rows, total input tokens and output tokens, and total monetary cost of LLM calls, allowing users to correlate the observed output with the optimization and execution choices visualized in Fig. 3b. When the user clicks **PROFILE**, the system will not only execute the query and display the results in SEMASTUDIO, but also visualize the optimization and execution details to be shown in §3.2. After query execution, the user is free to modify the configurations in Fig. 3a-④, and resubmit the query by **EXPLAIN**, **RUN**, or **PROFILE** to interactively compare the optimized plans and execution results.

3.2 Query Processing and Profiling

Fig. 3b visualizes the internal processing pipeline of SEMA, activated by clicking **PROFILE** (Fig. 3b-④). This panel presents how the submitted SEMASQL query is processed step by step and how intermediate plans flow across the optimizer and executor components to produce the final execution result.

LLM-assisted Expression Optimizations. The workflow begins with two LLM-assisted expression optimizations, *Expression Compression* in Fig. 3b-⑥ and *Predicate Deduction* in Fig. 3b-⑦. In Fig. 3b-⑥, all NL expressions extracted from the semantic operators are listed in the panel, followed by the corresponding compressed outputs generated by the Compression LLM in a parallel view, e.g., ‘Translated_Review is a complete sentence and has informative content.’ compressed into ‘Translated_Review is a complete, informative sentence.’, enabling direct comparison. A verification LLM then evaluates whether each compressed expression is semantically equivalent to its original form and reports the verdict for each case. The interface concludes by listing the validated compressed expressions together with runtime and cost metrics, thereby providing both process transparency and evidence of optimization effectiveness. In Fig. 3b-⑦, the interface lists the original NL predicates extracted from the *SemFilter* alongside the potential SQL predicates generated by predicate deduction, e.g., the SQL predicate ‘Translated_Review != “nan”’ deduced from the exemplar query in

Fig. 1 by the Deduction LLM and verified by the Reflection LLM. Along with the generated predicates, the interface displays the system prompts of the two auxiliary LLMs, total wall-clock time, and monetary cost of each LLM call.

Adaptive Query Execution. The optimized plan is then passed to the executor shown in Fig. 3b-⑧. Here, the system first generates multiple candidate execution paths, each corresponding to a distinct physical strategy for evaluating the semantic filters in the query, including operator reordering, fusion, and batching variants. The panel displays these paths side by side. The executor evaluates these paths over sample data and records their runtime statistics. The resulting measurements, including metrics such as latency, monetary cost, and the relative accuracy to the default path, are collected and displayed within each path node. These measurements are further summarized in Fig. 3b-⑨, where the interface renders a latency-cost scatter plot and displays the Pareto frontier in Fig. 3b-⑨. Each point corresponds to one candidate path, and only the non-dominated points (shown in red) are retained in the Pareto frontier (the red line). AQE then selects the best path according to the user-specified execution policy (latency-first or cost-first), highlighted as the yellow point in the figure. Once the policy is fixed, the selected path becomes the execution plan applied to the remaining data, and its executions result is then returned to the main result panel in Fig. 3a-⑤.

REFERENCES

- [1] Anas Dorbani, Sunny Yasser, Jimmy Lin, and Amine Mhedhbi. 2025. Beyond Quacking: Deep Integration of Language Models and RAG into DuckDB. *Proc. VLDB Endow.* 18, 12 (Aug. 2025), 5415–5418.
- [2] Chunwei Liu, Matthew Russo, Michael Cafarella, Lei Cao, Peter Baile Chen, Zui Chen, Michael Franklin, Tim Kraska, Samuel Madden, Rana Shahout, et al. 2025. Palimpsest: Optimizing ai-powered analytics with declarative query processing. In *Proceedings of the Conference on Innovative Database Research (CIDR)*. 2.
- [3] Liana Patel, Siddharth Jha, Melissa Pan, Harshit Gupta, Parth Asawa, Carlos Guestrin, and Matei Zaharia. 2025. Semantic Operators and Their Optimization: Enabling LLM-Based Data Processing with Accuracy Guarantees in LOTUS. *Proceedings of the VLDB Endowment* 18, 11 (2025), 4171–4184.
- [4] Kangkang Qi, Dongyang Xie, Wenbo Li, Hao Zhang, Yuanyuan Zhu, Jeffrey Xu Yu, and Kangfei Zhao. 2026. Sema: A High-performance System for LLM-based Semantic Query Processing. *arXiv:2603.11622 [cs.DB]*
- [5] Mark Raasveldt and Hannes Mühleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proc. SIGMOD*, Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 1981–1984.